

2025년 충북대학교 소프트웨어학과 데이터베이스 과제

DataBase System 보고서

- Flask Website Implementation -

2025년 충북대학교 소프트웨어학과 데이터베이스시스템 과제

(2021041072 전영우)

- 2025년 12월 5일 -

1. 프로그램 명세

본 프로그램은 아파트의 정보를 관리하기 위한 웹사이트입니다.
웹사이트의 기능은 다음과 같습니다.

- 1.1. 웹사이트의 **info** 탭을 통해 거주자(**resident**), 세대(**unit**), 차량(**vehicle**) 정보를 조회할 수 있습니다.
- 1.2. **Insert** 탭을 통해 각각의 테이블에 정보를 **insert** 할 수 있습니다.

2. Query문 설계

1. Insert문

```
sql = """
    INSERT INTO Resident (resident_id, First_name, last_name)
    VALUES (%s, %s, %s)
    """
```

```
sql = """
    INSERT INTO Building (building_id, construction_date, building_name, address)
    VALUES (%s, %s, %s, %s)
    """
```

```
sql = """
    INSERT INTO Unit (building_id, unit_id, exclusive_area, common_area)
    VALUES (%s, %s, %s, %s)
    """
```

```
sql = """
    INSERT INTO Vehicle (vehicle_number, vehicle_type, resident_id, building_id, registered_date)
    VALUES (%s, %s, %s, %s, %s)
    """
```

```
sql = """
    INSERT INTO Unit_Resident_relation (building_id, unit_id, resident_id)
    VALUES (%s, %s, %s)
    """
```

```
sql = """
    INSERT INTO Resident_PhoneNumber (resident_id, phone_number)
    VALUES (%s, %s)
    """
```

각 **table** 에 **data** 를 **inserting** 하기 위한 **insert문**입니다.

6개의 테이블이 있기 때문에 각각의 테이블에 정보를 넣어줄 **6**개의 **query문**을 작성하였습니다.

2. Info Searching Select문

2.1. Resident Info

```

if q: # 검색어가 있는 경우: 특정 사람만 조회
    # 숫자로만 입력한 경우 resident_id로 검색
    if q.isdigit():
        cursor.execute("""
            SELECT r.resident_id, r.First_name, r.last_name, p.phone_number
            FROM Resident r
            JOIN Resident_PhoneNumber p
            ON r.resident_id = p.resident_id
            WHERE r.resident_id = %s
            ORDER BY r.resident_id ASC
            """, (q,))
        residents = cursor.fetchall()

        total = len(residents)
        total_pages = 1

```

```

else:
    # 검색어 없으면 기존처럼 페이징 처리
    cursor.execute("""
        SELECT r.resident_id, r.First_name, r.last_name, p.phone_number
        FROM Resident r
        JOIN Resident_PhoneNumber p
        ON r.resident_id = p.resident_id
        ORDER BY r.resident_id ASC
        LIMIT %s OFFSET %s
        """, (limit, offset))
    residents = cursor.fetchall()

```

Resident Info 를 출력하는 쿼리문입니다.

휴대폰에 관한 테이블도 기존 **resident** 의 정보에 포함돼 있던 **Multi-valued** 이기 때문에, Join을 이용하여 합친 후 출력을 진행했습니다.

offset 을 이용하여 페이징 기능을 구현했습니다.

“(페이지 수 -1) * 페이지당 표현할 데이터 수” 공식을 이용하여 오프셋을 계산하고 그만큼 넘기면, 페이지 수만큼 데이터를 뛰어넘고 출력할 수 있습니다.

2.2. Unit Info Searching 문

```

if q: # 검색어가 있는 경우
    if q.isdigit():
        # 숫자면 building_id 또는 unit_id 기준 검색
        cursor.execute("""
            SELECT U.building_id, U.unit_id, U.exclusive_area,
            U.common_area, B.building_name
            FROM Unit U
            JOIN Building B ON U.building_id = B.building_id
            WHERE U.building_id = %s OR U.unit_id = %s
            ORDER BY U.building_id, U.unit_id
            """, (q, q))

```

```

else:
    # 문자열이면 건물 이름 + 그 유닛에 사는 사람 이름으로 검색
    like_q = f"%{q}%"
    cursor.execute("""
        SELECT
            U.building_id,
            U.unit_id,
            U.exclusive_area,
            U.common_area,
            B.building_name
        FROM Unit U
        JOIN Building B ON U.building_id = B.building_id
        WHERE
            B.building_name LIKE %s
            OR
            EXISTS (
                SELECT *
                FROM Unit_Resident_relation R
                JOIN Resident r2
                ON R.resident_id = r2.resident_id
                WHERE R.building_id = U.building_id
                AND R.unit_id = U.unit_id
                AND (r2.First_name LIKE %s OR r2.last_name LIKE %s)
            )
        ORDER BY U.building_id, U.unit_id
    """, (like_q, like_q, like_q))

```

```

cursor.execute("""
    SELECT U.building_id, U.unit_id, U.exclusive_area,
           U.common_area, B.building_name
    FROM Unit U
    JOIN Building B ON U.building_id = B.building_id
    ORDER BY U.building_id, U.unit_id
    LIMIT %s OFFSET %s
""", (limit, offset))
units = cursor.fetchall()

```

Unit table은 Building Table에 의존적인 Weak Entity입니다.
따라서, Building table과 Join을 통해 표현해 주었습니다.

검색어를 통해 특정 이름을 가진 resident가 사는 unit을 조회해 주기 위한 2중 쿼리문을 작성했습니다.

먼저 그 사람이 사는 unit의 unit_id를 들고 와야 하는데 이때, resident와 Unit은 M:N 관계 (한 사람은 여러 세대를 소유할 수 있고, 여러 사람이 하나의 Unit에 살 수 있기 때문) 이므로 Relation 테이블에서 Join을 통해 resident 정보를 연결하여 그 사람이 사는 세대의 unit_id를 먼저 탐색합니다.

이렇게 얻어낸 검색한 이름을 가진 unit의 id를 기반으로, Exist문을 활용해 Unit table에서 정보를 Select하면, 원했던 결과(ex) 민호 가 사는 세대 정보) 를 얻을 수 있습니다.

OR 연산자를 이용해 building_name LIKE를 묶어주면, 건물이름 혹은 거주인명 으로 정보를 검색할 수 있습니다.

2.3. Unit Info Searching 문

```

if q.isdigit():
    # 숫자면 vehicle_number, resident_id, building_id 쪽으로 검색
    cursor.execute("""
        SELECT
            vehicle_number,
            vehicle_type,
            resident_id,
            building_id,
            registered_date
        FROM Vehicle
        WHERE vehicle_number = %s
            OR resident_id = %s
            OR building_id = %s
        ORDER BY vehicle_number
    """, (q, q, q))

```

```

else:
    # 문자열이면 vehicle_type으로 LIKE 검색 (예: Sedan, SUV 등)
    like_q = f"%{q}%"
    cursor.execute("""
        SELECT
            vehicle_number,
            vehicle_type,
            resident_id,
            building_id,
            registered_date
        FROM Vehicle
        WHERE vehicle_type LIKE %s
        ORDER BY vehicle_number
    """, (like_q,))

vehicles = cursor.fetchall()
total = len(vehicles)
total_pages = 1 # 검색 결과는 한 페이지로

```

```

else:
    # 페이징 로직
    cursor.execute("""
        SELECT
            vehicle_number,
            vehicle_type,
            resident_id,
            building_id,
            registered_date
        FROM Vehicle
        ORDER BY vehicle_number
        LIMIT %s OFFSET %s
    """, (limit, offset))
    vehicles = cursor.fetchall()

```

위의 로직과 동일한 로직으로 검색을 진행합니다.

vehicle number, resident_id, building_id / vehicle_type 로 검색할 수 있습니다.

type은 string 형태이므로 따로 케이스를 분리했고 받아온 입력값을 바탕으로 LIKE, =(비교연산자)를 이용하여 검색이 가능합니다.